



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Processamento de Linguagens

Ano Letivo de 2025/2026

Construção de um Compilador para Fortran 77 Standard

Carlos Araújo

A106845

Vitor Senra

A106921

Hugo Soares

A106609

April 30, 2026

PL

Índice

1. Introdução	1
2. Arquitetura do Compilador	1
3. Análise Léxica	1
4. Análise Sintática	2
5. Análise Semântica e Tabela de Símbolos	2
6. Representação Intermédia (AST)	3
7. Otimizações e Geração de Código	3
7.1. Otimização (Middle-end)	3
7.2. Geração de Código VM (Back-end)	4
7.2.1. Mapeamento de Construções Fortran	4
8. Testes e Validação	4
9. Conclusões	5

1. Introdução

A disciplina de Processamento de Linguagens foca no estudo de compiladores e estruturas interpretativas. Este projeto consubstancia estes conhecimentos e implementa um compilador integral para a norma original clássica **Fortran 77**, que tem como base a geração estrita de instruções orientadas a uma Stack Virtual Machine (fornecida no âmbito curricular). O programa foi desenhado em **Python 3** recorrendo ativamente à biblioteca de *parsing* **PLY**.

O compilador segue e adota na íntegra a arquitetura standard de *compilers*, implementando um fluxo pipeline rigoroso modularizado em *Front-end* e *Back-end*: pré-processamento de fixed-formats, análise léxica para streams de tokens sintáticos, construção de Árvore AST, análise semântica acoplada por escopo em *Symbol Tables* e geração nativa de código .vm e otimizações. O presente relatório técnico descreve em detalhe os padrões, fluxos, restrições e as abordagens de *engineering* seguidas no projeto ao longo das sucessivas camadas.

2. Arquitetura do Compilador

O compilador possui módulos independentes em `src/`:

- `lexer.py`: Tokenização (via PLY).
- `parser.py`: Análise sintática e construção da AST.
- `ast_nodes.py`: Nós da árvore abstrata usando `@dataclass`.
- `symbol_table.py`: Tabela de símbolos com escopo.
- `optimizer.py`: Otimizações na AST (Constant Folding, Dead Code Elimination).
- `codegen.py`: Geração de instruções VM (Stack machine).

O `main.py` orquestra o pipeline completo: lê .for, gera tokens, constrói a AST, otimiza a árvore, e emite código para um ficheiro .vm. A separação em fases (Front-end, Middle-end/Optimizer, Back-end) garante uma arquitetura modular, fácil de testar (via `pytest`) e manter.

3. Análise Léxica

A análise léxica é o primeiro passo da conduta, garantindo a decomposição da fonte Fortran num fluxo contínuo de *tokens* validados. Desenvolvida no módulo `lexer.py` usando `ply.lex`, garante o tratamento nativo do **Fixed-Format** do Fortran 77:

- **Colunas 1-5**: Reservadas unicamente para numeração de identificação local de instruções (Labels/GOTO targets).
- **Coluna 6**: Reservada como caracter de controlo de contiguidade de linha ou denotação de comentário (quando C ou *).
- **Colunas 7-72**: Espaço principal para a escrita do corpo sintático.

A nível preparatório, o pré-processador interno normaliza o source *in memory* para remover os comentários explícitos e unir quebras formatadas contíguas.

Os Tokens emitidos para o Parser incluem as *keywords* base da norma (PROGRAM, IF, DO, CONTINUE), operadores lógicos nativos e transacionais (.GT., .AND., **), literais de múltiplas tipagens (Inteiros, Reais com formato E, literais String entre aspas isoladas, ou lógicos como .TRUE.). Como estipulado, todos os identificadores (tendo tamanho limite de 6 caracteres alfanuméricos começados por letra) são recolhidos e sistematicamente transformados em letras maiúsculas (.upper()), respeitando a **Case-Insensitivity** fundamental do Fortran 77. Em caso de divergência ortográfica na formação, o lexer atira uma `LexerError` reportando a posição.

4. Análise Sintática

A análise sintática baseou-se na implementação rigorosa da Context-Free Grammar (CFG) desenvolvida para a norma em questão (ver anexo em formato `CFG.md`). A sua base é LALR(1) suportada diretamente pelas implementações estritas da engine `PLY Yacc` via módulo `parser.py`.

A priorização operacional de expressões avaliadas obedece estritamente às tabelas da linguagem: primeiramente operações lógicas, relacionais, até à base aritmética (`logical_or` → `logical_and` → `relational` → `additive` → `multiplicative` → `power` → `unary` → `primary`).

Concomitantemente à validação gramatical da cadeia, o módulo sintático executa e gere chamadas assíncronas à infraestrutura Semântica, parando ou garantindo o avanço se as regras de alocação se provarem válidas (variáveis declaradas, acessos dentro de arrays declarados como limitados a 1 Dimensão, blocos aninhados e `GOTO` matches). No final da verificação descendente com sucesso em cada ramificação da regra gramatical (`p_[rule]`), instanciam-se *Dataclasses* limpas injetáveis para a construção integral da **Abstract Syntax Tree (AST)**. Em caso de divergência estrita (como falta de parentesis e terminação incorreta de loop `DO`), é injetado o respetivo lançamento para o standard output em formato legível de `SyntaxError`.

5. Análise Semântica e Tabela de Símbolos

Durante a passadeira sintática LALR(1), o compilador invoca assíncronamente os métodos expostos na sua **Tabela de Símbolos**. O `symbol_table.py` materializa-se como uma pilha encadeada LIFO suportando um modelo rigoroso de Nested Scopes (Escopos Aninhados). No entanto, o global assume prioridade total com reuso partilhado quando em escopo principal e declaração estrita via Subprogramas, alocando a cada nó global uma *Dataclass* `Symbol` que preserva toda a metainformação crucial do código-fonte (Tipagem - se Integer, Logical, Array dimensional com indexação de limites, número de linhas e origens de declaração).

A validação de conflitos (lançada internamente na aplicação como *exceptions* de natureza tipológica `SemanticError`) ocorre de maneira natural perante as regras:

- Verificação restrita de re-declarações (`declare()`), não permitindo sombras ou conflitos em variáveis na mesma *frame*.
- Obrigatoriedade de pré-declaração em invocações implícitas (`lookup()`).
- O acesso fora de limite em dimensões (index bounds restritos) por sub-rotinas array-oriented (`check_array_access()`).
- Casting implícito ou limitação inter-tipológica: o motor suporta a limitação natural do Fortran, no qual a alocação REAL num bloco associado de memória INTEGER gera truncação intrínseca, mas LOGICAL para tipos matemáticos acusa colisão imediata.

6. Representação Intermédia (AST)

A estruturação final das validações recai na geração da **Abstract Syntax Tree (AST)**. Em oposição às implementações baseadas puramente em tuplos nativos em Python, adotou-se o uso estrito de módulos `@dataclass` (`ast_nodes.py`), que propiciam uma semântica rígida, documentada e com verificação de tipo.

A infraestrutura ramifica-se nos seguintes nós de topo:

- **CompilationUnit** e **Program**: O ponto fulcral e originário da representação para o main script e subprogramas.
- **Declarações**: Suportadas via `TypeDeclaration` e parâmetros globais imutáveis.
- **Statements (Instruções)**: Enquadramentos complexos de fluxo via `IfThenElse` (com suporte a ramificações múltiplas de *elseif*), `DoLoop`, saltos explícitos via `GotoStatement`, e as primitivas IO `ReadStatement` / `PrintStatement`.
- **Expressões (Expressions)**: Cálculos puramente avaliados via `BinaryOp`, `UnaryOp`, e resoluções terminadas em `Literal` ou chamadas recursivas `FunctionCall`.

Todos os nós estendem rigorosamente uma classe base abstracta genérica `Node`, na qual o rastreamento da linha (`lineno`) garante depuração clara caso a *Pipeline* intercetar falhas na Geração ou Otimização. Em última análise, a AST limpa e validada garante o isolamento funcional do Front-end em relação ao Back-end.

7. Otimizações e Geração de Código

7.1. Otimização (Middle-end)

Antes de gerar código, a Árvore Sintática Abstrata (AST) passa por um módulo de otimização (`optimizer.py`). Implementámos passagens de otimização focadas na melhoria de performance (valorização), nomeadamente:

- **Constant Folding**: Expressões binárias (+, -, *, /) entre literais numéricos são avaliadas em tempo de compilação.
- **Dead Code Elimination (DCE)**: Remoção de código inatingível (e.g. instruções após um `GOTO` ou `RETURN` que não possuam `label`).

7.2. Geração de Código VM (Back-end)

O módulo `codegen.py` implementa o padrão **Visitor** para percorrer a AST e gerar um ficheiro `.vm` com as instruções para a máquina virtual baseada em pilha (stack machine).

7.2.1. Mapeamento de Construções Fortran

- **Variáveis e Atribuições:** Utiliza-se um contador global de endereços. A cada variável não mapeada é atribuído um offset. A escrita faz-se com `STOREG addr` e a leitura com `PUSHG addr`. Arrays usam instruções correspondentes (`STOREA`).
- **Aritmética e Lógica:** Expressões são avaliadas em pós-ordem. Operadores geram as instruções nativas: `ADD`, `SUB`, `MUL`, `DIV`, `INFEQ`, `SUPEQ`, etc.
- **Controlo de Fluxo (IF / GOTO):** As *labels* de Fortran (inteiros 1-5 colunas) são mapeadas para *labels* VM unívocas (ex: `L0`, `L1`). Instruções como `JZ` (Jump if Zero) e `JUMP` realizam os desvios.
- **Ciclos (DO):** Um `DO loop DO 10 I=1, N` inicializa a variável de iteração, define a *label* de início, testa a condição (saída via `JZ`), corre o corpo, incrementa a variável, e faz o `JUMP` para o início.
- **I/O:** `READ` extrai o input e armazena via `STOREG`. `PRINT` coloca as expressões na pilha e emite `WRITEI`, `WRITELN` ou `WRITES`.

8. Testes e Validação

O compilador foi validado de ponta a ponta com cinco programas baseados nos cenários de teste da disciplina. Para automatizar este processo, foi desenvolvida uma *test suite* em Python utilizando a biblioteca `pytest` (`tests/test_compiler.py`).

A automação garante a verificação contínua (CI) e testa os seguintes programas:

- `hello.for`: Avalia rotinas básicas de I/O (`PRINT` de *strings*).
- `fatorial.for`: Valida o cálculo matemático, controlo de fluxo complexo iterativo (`DO`) e resolução de variáveis escalares inteiras.
- `primo.for`: Verifica ciclos, manipulação lógica e construções condicionais (`IF/GOTO`) e ramificações condicionais baseadas em booleanos implícitos.
- `somaarr.for`: Testa o correto parseamento de dimensões, declarações e acesso por indexação a Arrays Unidimensionais (1D).
- `conversor.for`: Valida a compatibilidade e a transição entre múltiplos subprogramas (Funções e Procedimentos).

Em todos os casos, a suite não só avalia o sucesso da fase léxica e sintática (sem levantar erros do parser/lexer) mas constrói a AST, realiza as otimizações Middle-end via `optimizer.py`, e efetivamente emite o binário (`.vm`). Todos os 5 testes garantem retorno de código de erro zero.

9. Conclusões

Este projeto demonstrou, com sucesso, a implementação rigorosa de um compilador de uma linguagem rica, como Fortran 77, em Python. Cumpriram-se os requisitos propostos com distinção. A modularidade (via Visitor e abordagens pipeline) facilitou a manutenção e extensibilidade.

As principais dificuldades envolveram o tratamento estrito de *labels* via formatação fixa clássica e o mapeamento fiel em arquitetura stack-machine. Outro aspeto desafiante consistiu em garantir que nós AST não-tradicionais fossem resolvidos e alocados uniformemente sem dependências cruzadas (resolvido por resolução de “post-order”).

O trabalho supera a cota de 10/20 ao realizar alocação com a Tabela de Símbolos, passagens bônus como Constant Folding e DCE via `optimizer.py`, garantindo performance para a VM. Como trabalho futuro propomos a integração de Common Subexpression Elimination através da geração local de um Control Flow Graph. Em suma, desenvolveu-se uma ferramenta robusta e academicamente completa.